

Z2 升班测 解析

T1 蛋糕与甜甜圈

难度： 入门

解析

鸡兔同笼问题的变种，考察基础枚举，双重循环分别枚举蛋糕和甜甜圈数量即可。

参考代码

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
    int a,b,n;
    cin>>a>>b>>n;
    int ans=0;
    for(int i=0;i<=n;i++)
    {
        for(int j=0;j<=n;j++)
        {
            if(a*i+b*j==n)
            {
                ans++;
            }
        }
    }
    cout<<ans;
    return 0;
```

```
}
```

T2 实时名次更新

难度：入门，考察内容：题意理解

解析

不难发现，只有当输入的成绩比小明成绩高的时候，小明的名次才会下降，如果输入的成绩比小明的成绩低，那么是排在小明的后面，小明前面的人的数量是不会变的，所以此时小明的名次不会变化。

因此，设置一个初始名次 `num = 1`，若输入的 `a[i] > a[0]`，则直接让 `num++` 即可，否则不变。

参考代码

```
#include<iostream>
#include<algorithm>
#include<cstdio>
using namespace std;

int a[200005];

int main()
{
    int n;
    cin >> n;
    for (int i = 0; i <= n; i++)
    {
        cin >> a[i];
    }
    int num = 1;
    for (int i = 1; i <= n; i++)
    {
        if (a[i] > a[0])
```

```
{
    num++;
    cout << num << ' ' << "down" << endl;
}
else cout << num << ' ' << "-" << endl;
}
return 0;
}
```

T3 Substring

难度：普及-，考察内容：字符串string

题目大意

给定两个字符串 s, t ，问：至少修改 s 中的几个字符，能够使得字符串 t 是 s 的子串。

解析

由于题目中只允许修改，不允许增加或删除，因此必须从字符串 s 中找到一个长度与 t 相等的子串，并且该子串中与 t 中的不同的字符数量尽可能少。

于是，我们可以遍历字符串 s ，选取该位置为子串的起点，然后逐一截取子串即可。

注意：起点为 i ，那么终点为 $i + t.size() - 1$ ，需要保证终点 $< s.size()$ 。因此，循环范围为： $i = 1 \sim s.size() - t.size() + 1$

参考代码

```
#include<iostream>
#include<algorithm>
using namespace std;
int ans[1005],minn=2e4;
int main()
```

```
{
    string s,t;
    cin>>s>>t;
    for(int i=0; i<s.size()-t.size()+1;i++)
    {
        for(int j=0;j<t.size();j++)
        {
            if(s[i+j]!=t[j]) ans[i]++;
        }
        minn=min(minn,ans[i]);
    }
    cout<<minn;
}
```

T4 图像压缩

难度：普及-，考察内容：二维数组

题目大意

给定一个二维数组，其中分为“黑色”，“白色”两种值，若某一行内所有元素均为白色，就可以删除这一行。同理，对于某一列，如果均为白色，也可以删除这一列。输出在这些操作都执行完之后的图像。

解析

我们换一个角度去思考，不要去考虑删除。如果 $a[i][j]$ 为黑色，那么不可能对第 i 行和第 j 列进行统一的删除，这一行和这一列就需要留下，由此可得到每一行和列是否要留下。可以建立两个数组， $h[i]$ 表示第 i 行是否要留下， $l[j]$ 表示第 j 列是否要留下。在有了这两个数组后，我们就可以通过这两个数组来判断任意一个元素是否被删除。只有 $h[i] \&\& l[j]$ 的 $a[i][j]$ 才需要输出。

参考代码

```
#include<iostream>
```

```
using namespace std;

const int H = 1e3+10, W = 1e3+10;

int n, m;
char image[H][W];
bool h[H], l[W];

int main(){
    cin >> n >> m;
    for(int i=1; i<=n; i++)
    {
        for(int j=1; j<=m; j++)
        {
            cin >> image[i][j];
            if(image[i][j] == '#')
            {
                h[i] = true;
                l[j] = true;
            }
        }
    }

    for(int i=1; i<=n; i++)
    {
        for(int j=1; j<=m; j++)
        {
            if(h[i] && l[j]){
                cout << image[i][j];
            }
        }
        if(h[i]) cout << endl;
    }

    return 0;
}
```

T5 Peak

难度：普及-，考察内容：队列窗口、双指针

题目大意

皮皮收到了 n 个礼物，每个礼物的编号为 a_i （编号可能重复）。皮皮可以选择其中的一部分，选择的部分必须保证在区间 $[x, x + m)$ 之内（可以选 x ，但是不可以选 $x + m$ ）。 x 的值可以自己确定，问：皮皮最多能选走多少个礼物。

即给定 n 个数，问在长度为 m 的区间内，最多有几个数。

解析

区间的长度是固定的，问区间内部最多几个数，这种问题就是标准的队列窗口问题。固定了长度，那么只需要将区间从最左侧逐一往右移动即可。在移动过程中就会发现，最先进入区间的，一定是最先离开的，满足队列的性质。

队列窗口的核心代码：

```
for (int i = 1; i <= n; i++)
{
    while (q.size() && a[i] - q.front() >= m)
        q.pop();

    q.push(a[i]);
    ans = max(ans, int(q.size()));
}
```

参考代码

```
#include <bits/stdc++.h>
using namespace std;

int n, m, a[int(3e5 + 1)], ans;
queue<int> q;
```

```

int main()
{
    cin >> n >> m ;
    for(int i = 1 ; i <= n ; i++)
        cin >> a[i] ;

    sort(a + 1 , a + n + 1) ;

    for (int i = 1 ; i <= n ; i++)
    {
        while (q.size() && a[i] - q.front() >= m)
            q.pop();
        q.push(a[i]);
        ans = max(ans, int(q.size()));
    }

    cout << ans << endl ;
    return 0 ;
}

```

T6 简单背包2

题目解析

标准的 **dfs** 题目，具体参考代码

参考代码

```

#include<iostream>
using namespace std;

int k, W, n[15], w[15], v[15], ans = 0;

```

```

void dfs(int step, int weight, int value)
{ //接下来要搜第step类物品，前面已经放入背包的物品重量weight、价值value
  if(step == k + 1)
  {
    ans = max(ans, value); // 更新最大价值
    return; // 如果所有物品类型都已经考虑完毕，返回
  }

  // 逐一尝试选取 0 到 n 个当前物品类型
  for(int i = 0; i <= n[step]; i++)
    if(weight + i * w[step] <= W)
      dfs(step + 1, weight + i * w[step], value + i * v[step]);
}

int main()
{
  cin >> k >> W;

  for(int i = 1; i <= k; ++i) cin >> n[i] >> w[i] >> v[i];

  dfs(1, 0, 0);

  cout << ans << endl;

  return 0;
}

```

T7 余数最大子段和

解析

60分代码较为容易，不多说。主要说一下满分方法。

我们希望： $(sum[i] - sum[j - 1]) \% k == 0$ 因此，可以化简出：

$$sum[i] \% k == sum[j - 1] \% k$$

进而发现，其实要保证子段和最大，只需要用当前的减去对 k 同余的前缀和中最小的一个即可。即让上式中的 $sum[j - 1]$ 尽可能小。

参考代码

```
#include<iostream>
#include<cstring>
using namespace std;

int n, k, a[100005], s[100005], min_s[100005], ans;

int main()
{
    cin >> n >> k;
    for(int i = 1; i <= n; i++)
    {
        cin >> a[i];
        s[i] = s[i-1] + a[i];
    }

    memset(min_s, 0x3f, sizeof(min_s)); //初始化最小值为极大值
    for(int i = 1; i <= n; i++)
    {
        int r = (s[i] % k + k) % k; // 计算余数，同时防止负余数
        ans = max(ans, s[i] - min_s[r]);
        min_s[r] = min(min_s[r], s[i]); //找到余数的最小值
    }
    cout << ans << endl;

    return 0;
}
```